

Reviewer Pack — Minimal Rank of Quandle Invariants vs AC^0 -k-Distance Circuit Complexity

Entry 4bb9368c2e10 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 10:14:54 UTC

Minimal Rank of Quandle Invariants vs AC^0 -k-Distance Circuit Complexity

Entry ID: 4bb9368c2e10 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 10:14:54 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: AC^0 -k-Distance Circuit Complexity

Statement:

For any boolean function f with n variables, the minimal rank of its associated quandle invariant $Q(f)$ is polynomially related to the minimum size of an AC^0 -k-distance circuit that computes f . Specifically, there exists a constant $c > 0$ such that for all n , the minimal rank of $Q(f)$ is at most $cn \log(n)$.

Rationale (proposer's reasoning):

Quandles provide a combinatorial algebraic structure that can capture non-trivial properties of boolean functions. The conjecture aims to link this abstract structure with concrete complexity measures, potentially revealing new insights into circuit complexity lower bounds.

Taxonomy category: Quandle_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5e2867fe2219e913

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all boolean functions f with n variables, the minimal rank of $Q(f)$ is at most $c * n * \log(n)$, where c is a polynomially bounded constant and $n \log(n)$ is calculated using an aggregate statistic over multiple seeds. The conjecture is falsified if any function f has a minimal rank of $Q(f)$ greater than $c * n * \log(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_quandle_invariant(f):
        n = int(math.log2(len(f)))
        q = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            for j in range(n + 1):
                if (i <= n) | (1 <= j) - 1 < len(f):
                    q[i][j] = f[(i <= n) | (1 <= j) - 1]
        return q

    def min_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(min(m, n)):
            if matrix[i][i] == 0:
                found_pivot = False
                for k in range(i + 1, m):
                    if matrix[k][i] != 0:
                        matrix[i], matrix[k] = matrix[k], matrix[i]
                        found_pivot = True
                        break
                if not found_pivot:
                    continue
            pivot = Fraction(matrix[i][i])
            for j in range(n):
                matrix[i][j] /= pivot
            for k in range(m):
                if k != i and matrix[k][i] != 0:
```

```

        factor = -matrix[k][i]
        for j in range(n):
            matrix[k][j] += factor * matrix[i][j]
    rank += 1
    return rank

def min_ac0_k_distance_circuit_size(f):
    n = int(math.log2(len(f)))
    # Placeholder for actual AC^0-k-distance circuit size computation
    # This is a dummy implementation that returns a constant value
    return n * (n + 1) // 2

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):
        f = generate_boolean_function(n)
        q = compute_quandle_invariant(f)
        rank = min_rank(q)
        ac0_k_distance_circuit_size = min_ac0_k_distance_circuit_size(f)
        if rank > n * math.log2(n) * 1.1: # Adding a small margin for polynomial relation
            conjecture_holds = False
            counterexample = f"n={n}, rank={rank}, ac0_k_distance_circuit_size={ac0_k_distance_circuit_size}"
        total_metric_value += rank
        instances_tested += 1

return {
    "metric_name": "Minimal Rank",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing={results[0]['seed']}")

```

```

else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered criteria. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10544
2	preregistration	ollama_remote	glm4:latest	0	0	5714
3	novelty	ollama_remote	glm4:latest	0	0	4748
4	novelty	ollama_remote	glm4:latest	0	0	5166
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11405
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7027
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10856
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11383
9	judge	ollama_remote	glm4:latest	0	0	9026

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 75870 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4bb9368c2e10.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4bb9368c2e10.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4bb9368c2e10.tar.gz (if generated)